# CAREER COMPASS AI

## Complete Interview-Ready Technical Documentation

**Project Subtitle:** AI Skill Gap & Resume Analyser | Full Codebase Architecture & Technical Interview Defense

---

## 1. PROJECT OVERVIEW

**Project Title:** AI Skill Gap and Resume Analyser / Career Compass AI
**Problem Statement:** Job seekers and students struggle to align their resumes with dynamic industry Job Descriptions (JDs). ATS systems reject up to 75% of resumes due to keyword mismatches, formatting flaws, and unquantified impact. Candidates lack objective, data-driven feedback on exact skill gaps, domain readiness, and structured learning paths.

**Why This Problem Matters:** Without automated analysis, candidates waste hundreds of hours applying blindly without knowing why their applications fail. Bridging this gap requires objective text processing, skill classification, ATS scoring, and targeted roadmap generation.

**Target Users:** Job seekers, university students, career switchers, technical recruiters, and educational institutions.

**Main Objective:** Provide an end-to-end web platform that parses PDF resumes, extracts skills via rule-based NLP, evaluates Job Descriptions, computes weighted ATS & skill gap scores, generates domain readiness metrics, and provides adaptive learning roadmaps and project recommendations.

**Key Features Implemented:**
- **Multi-Page PDF Parsing:** Client-side (PDF.js) and Server-side (PyMuPDF fitz) text extraction.
- **Rule-Based NLP Skill Extraction:** Normalization & alias dictionary matching across 100+ technical skills.
- **Weighted ATS Scoring Engine:** Evaluates Format (20%), Experience (25%), Skills (30%), Education (10%), & Impact (15%).
- **Job Description Classification:** Distinguishes Required vs. Preferred skills using contextual heuristics.
- **Skill Gap & Priority Calculation:** Computes coverage %, job match %, and prioritizes missing skills (High/Med/Low).
- **Domain Fit & Career Readiness:** Evaluates readiness across software domains (Frontend, Backend, Data Science, etc.).
- **Interactive Assessment & Analytics:** Role-based technical quiz engine with historical attempt tracking.
- **Adaptive Roadmaps & Project Recommender:** Maps missing skills to actionable projects and learning timelines.

**Software Development vs. AI/Data Science Breakdown:**
- **Software Engineering (Full-Stack):** React 19, TypeScript, Vite, Wouter routing, Tailwind CSS, Zustand state stores, Recharts visualization, FastAPI server, SQLAlchemy ORM, SQLite database.
- **AI / Data Science (NLP & Analytics):** PyMuPDF text parsing, regex normalization, entity keyword matching, heuristic skill classification, weighted scoring vector algorithms, domain fit analytics.

**INTERVIEW ELEVATOR PITCHES:**
• **30-Second Pitch:** "Career Compass AI is an intelligent resume analyzer and skill gap detection platform. It parses candidate resumes, extracts technical skills using NLP text normalization, compares them against job descriptions, and calculates an objective ATS score and skill gap matrix to provide custom learning roadmaps."
• **1-Minute Pitch:** "I developed Career Compass AI to solve high ATS rejection rates for job seekers. The system extracts text from PDF resumes using PyMuPDF, normalizes terms to resolve alias variations, and extracts skills across 100+ technical domains. It then analyzes JDs to classify required vs. preferred skills, computes a weighted ATS score based on format, impact, experience, education, and skills, and generates domain readiness scores alongside targeted project recommendations."
• **2-Minute Pitch:** "Career Compass AI is a full-stack AI/Data Science application built with React 19, TypeScript, FastAPI, and Python NLP services. When a candidate uploads a PDF resume, our dual-parsing layer extracts raw text, applies regex normalization, and executes dictionary pattern matching to identify candidate skills. On the job side, our JD engine classifies skills into required and preferred based on contextual text heuristics. The candidate-to-JD engine performs weighted set matching, producing an ATS score breakdown, skill gap priority queue, and domain readiness radar. Furthermore, candidates can take adaptive technical assessments to benchmark their proficiency and receive customized project roadmaps."

## 2. TECHNOLOGY STACK — ACTUAL PROJECT

| Technology | Where Used | Why Used | Alternative | Why This Choice |
|---|---|---|---|---|
| React 19 | client/src | Component UI architecture | Vue / Angular | Ecosystem & Virtual DOM speed |
| TypeScript 5.6 | client/src | Strict type safety | Plain JS | Prevents runtime state bugs |
| Vite 7.1 | Root / client | Fast HMR build tool | Webpack / CRA | Instant dev server start |
| Tailwind CSS 4 | client/src/index.css | Utility-first styling | Bootstrap | Custom dark glassmorphism design |
| Wouter 3.3 | client/src/App.tsx | Lightweight router | React Router v6 | Minimal footprint (<1.5KB) |
| Zustand 5.0 | client/src/state | Global state management | Redux Toolkit | Boilerplate-free hook state |
| Recharts 2.15 | client/src/components/charts | Data visualization | Chart.js | Native React SVG support |
| PDF.js 3.11 | client/src/lib/pdf.ts | Client PDF text extract | pdf-parse | Client-side browser parsing |
| Python 3.11 / FastAPI | server/main.py | High-perf REST backend | Node.js Express | Native NLP & ML library support |
| PyMuPDF (fitz) | server/services/resume_parser.py | Server PDF extraction | PyPDF2 / pdfplumber | 10x faster execution speed |
| SQLAlchemy | server/core/database.py | Python Database ORM | Raw SQL | Type-safe database abstraction |
| SQLite / PyMongo | server/career_compass.db | Local relational DB | PostgreSQL | Zero-config local persistence |

## 3. COMPLETE FOLDER STRUCTURE

**Project Codebase Directory Layout:**

• **server/main.py:** FastAPI entry point, CORS setup, router inclusion (`/api/auth`, `/api/resumes`, `/api/job-descriptions`, `/api/assessments`).
• **server/services/resume_parser.py:** PDF byte parser using PyMuPDF (`fitz`), clean text regex processing.
• **server/services/skill_extractor.py:** Text normalization and alias keyword matching engine.
• **server/services/resume_analyzer.py:** ATS scoring engine (5 signals), domain fit calculator, scorecard generator.
• **server/services/jd_analyzer.py:** Job Description title extraction, required vs preferred skill classifier.
• **server/data/skills.json & domains.json:** Canonical skills database and target domain skill mappings.
• **client/src/App.tsx:** Wouter client routing table and QueryClientProvider wrapper.
• **client/src/lib/pdf.ts:** PDF.js browser text extraction service with progress callback.
• **client/src/lib/analyzer.ts:** Client-side resume analysis fallback engine.
• **client/src/lib/skill-matcher.ts:** Candidate vs. JD skill comparison & priority sorting.
• **client/src/state/*.ts:** Zustand global state stores (`resume-store`, `job-store`, `assessment-store`, `auth-store`).
• **client/src/pages/*.tsx:** 15 full UI views (`dashboard`, `resume-upload`, `ats-dashboard`, `job-analysis`, `skill-gap-analysis`, `assessment`, `analytics`, `learning-roadmap`, `readiness`).

## 4. COMPLETE END-TO-END FLOW

**Step-by-Step Data Flow (User Upload to Dashboard):**

1. **User Upload:** Candidate selects a PDF file via drag-and-drop on `client/src/pages/resume-upload.tsx`.
2. **Client Processing & API Dispatch:** File is validated (<10MB, PDF format). Client uses `pdf.ts` (PDF.js) or dispatches `POST /api/resumes/upload` with FormData via `api.ts`.
3. **Text Extraction:** PyMuPDF (`fitz.open`) reads bytes in `resume_parser.py`, extracts page strings, and raises error if scanned/encrypted.
4. **Text Preprocessing:** `clean_resume_text` normalizes whitespace, tabs, and line breaks while preserving technical characters (+, #, .).
5. **Skill Extraction & Normalization:** `extract_skills` normalizes text to lowercase, strips punctuation, and matches against `skills.json` names and alias arrays.
6. **JD Analysis & Classification:** `jd_analyzer.py` parses JD text, identifying required skills (near "must have") vs preferred skills (near "nice to have").
7. **Candidate vs. JD Comparison:** `matchResumeToJd` (`skill-matcher.ts`) filters matching vs missing skills and computes priority levels.
8. **ATS Scoring & Domain Fit:** `compute_ats` evaluates Format, Experience, Skills, Education, & Impact. `compute_domain_fits` computes radar chart vector scores.
9. **Recommendation & Roadmap:** Missing skills feed `project-recommender.ts` and `roadmap.ts` to build personalized career steps.
10. **UI Render:** Zustand stores update and React components render scorecards, Recharts radar/bar charts, and skill gap tables.

## 5. RESUME PDF PROCESSING

**Accepted File Types:** `.pdf` (validated via file extension and `content_type == 'application/pdf'`).
**Client PDF Extraction:** Implemented in `client/src/lib/pdf.ts` using `pdfjs-dist/legacy/build/pdf`. ArrayBuffer is loaded via `pdfjsLib.getDocument`, iterating through pages and assembling text content.
**Backend PDF Extraction:** Implemented in `server/services/resume_parser.py` using PyMuPDF (`fitz`). Reads raw bytes with `fitz.open(stream=pdf_bytes, filetype="pdf")`.
**Error Handling & Edge Cases:**
• Encrypted PDF -> Raises `400 Bad Request` ("Password-protected PDF files are not supported").
• Zero pages / Empty file -> Raises `400 Bad Request` or `413 Payload Too Large` if >10MB.
• Scanned / Image-only PDF -> Detects empty text string post-parsing and raises `400 Bad Request` ("Could not extract text from PDF. The file may be image-only or scanned without OCR text").

**Tanglish Explanation:** PDF file-a open panni, PyMuPDF load panno. Pages illana or password protection irundha immediate-a error raise panno. Scanned images-ah irundha text extract aagadhu, so validation throw panno.

**Interview Defense Strategy:** "We chose PyMuPDF on the backend because it runs natively in C (MuPDF engine), making it 10x faster than pure-Python parsers like PyPDF2. On the frontend, we integrated PDF.js so candidates receive immediate feedback even before backend upload."

## 6. NLP / TEXT PROCESSING

**Techniques Implemented:**
• **Tokenization & Lowercasing:** All text is converted to lowercase via `text.lower()`.
• **Punctuation Cleaning with Character Preservation:** `re.sub(r'[^a-z0-9+.#\s-]', ' ', text)` cleans special symbols while preserving critical tech tokens like `C++`, `C#`, `.NET`, `Node.js`.
• **Whitespace Normalization:** `re.sub(r'\s+', ' ', text)` collapses multi-spaces and newlines into uniform single spaces.
• **Regex Pattern Matching:** Used in `jd_analyzer.py` for experience detection (`re.search(r'\d+\+?\s*years', text)`) and education degree detection (`re.search(r'bachelor|master|phd', text)`).

**Critical AI Distinction:** This project utilizes **Rule-Based NLP and Heuristic Text Analytics**. It does NOT use trained deep learning weights or LLM transformers, guaranteeing 100% deterministic, instant, and reproducible results.

## 7. SKILL EXTRACTION

**Skill Extraction Logic:**
Skills are extracted using `extract_skills()` in `server/services/skill_extractor.py` and `client/src/lib/analyzer.ts`. The system loads `skills.json`, where each skill defines a canonical name and an array of alias keywords.

**Concrete Code Trace & Example:**
• *Resume Text:* "Experienced in React.js, Java, Python, and MySQL."
• *JD Text:* "Required: React, Java, Docker, AWS."
1. Text is normalized: `"experienced in react.js java python and mysql"`
2. Skill loop evaluates `React`: Canonical name = `"React"`, Keywords = `["React", "React.js", "ReactJS"]`. Normalized keyword `"react.js"` is found in text -> Match added: `"React"`.
3. Output: `["Java", "MySQL", "Python", "React"]` (Sorted alphabetically, deduplicated).

## 8. SKILL NORMALIZATION

| Raw Skill Input | Normalized Text | Canonical Skill Output |
|---|---|---|
| React.js / ReactJS / React JS | react js | React |
| Node.js / NodeJS / Node | node js | Node.js |
| Python / python / PYTHON | python | Python |
| Amazon Web Services / AWS | aws | AWS |
| PostgreSQL / Postgres / PG | postgres | PostgreSQL |

## 9. SKILL MATCHING ALGORITHM

**Matching Engine (`client/src/lib/skill-matcher.ts`):**
• **Matched Skills:** $S_{\text{matched}} = S_{\text{resume}} \cap S_{\text{JD}}$
• **Missing Skills (Required):** $S_{\text{missing}} = S_{\text{JD, required}} \setminus S_{\text{resume}}$
• **Improvement Areas (Preferred):** $S_{\text{improve}} = S_{\text{JD, preferred}} \setminus S_{\text{resume}}$

**Priority Sorting:**
• *High Priority:* Skill is missing AND JD Importance $\ge 8$.
• *Medium Priority:* Skill is missing AND JD Importance $\ge 5$.
• *Low Priority:* Skill is preferred or optional.

**Time Complexity:** $O(N \times M)$ where $N$ is candidate skills and $M$ is JD skills. With normalized set hash lookups, matching reduces to $O(N + M)$.

## 10. SKILL GAP CALCULATION

**Formulas:**
1. **Required Skill Coverage:**
$$\text{Skill Coverage (\%)} = \left( \frac{|S_{\text{matched required}}|}{|S_{\text{total required}}|} \right) \times 100$$
2. **Weighted Job Match Score:**
$$\text{Job Match Score (\%)} = \left( \frac{\sum_{s \in S_{\text{matched}}} \text{Importance}(s)}{\sum_{s \in S_{\text{all JD}}} \text{Importance}(s)} \right) \times 100$$

**Numerical Worked Example:**
• Total JD Required Skills = 10. Matched Required Skills = 7.
• Skill Coverage = $(7 / 10) \times 100 = 70\%$.
• Total Importance Weight = 80. Matched Importance Weight = 60.
• Job Match Score = $(60 / 80) \times 100 = 75\%$.

## 11. ATS / RESUME SCORE

**Weighted Scoring Signals (`server/services/resume_analyzer.py` & `client/src/lib/analyzer.ts`):**
• **Format Score (20% Weight):** Checks presence of 8 section headers (`email`, `phone`, `linkedin`, `github`, `experience`, `education`, `skills`, `projects`).
• **Experience Score (25% Weight):** Checks industry terms (`intern`, `engineer`, `developer`, `analyst`, `lead`, `manager`, `team`).
• **Skills Score (30% Weight):** Calculated as $\min(100, \max(20, \text{Coverage} \times 1.25))$, measuring hits against top 26 skills in database.
• **Education Score (10% Weight):** Checks degree terms (`bachelor`, `master`, `b.sc`, `m.sc`, `phd`, `university`).
• **Impact Score (15% Weight):** Checks metrics and action verbs (`%`, `increased`, `reduced`, `improved`, `optimized`, `launched`, `saved`).

**Formulas:**
$$\text{ATS Score} = 0.20 \cdot F + 0.25 \cdot E_{\text{exp}} + 0.30 \cdot S + 0.10 \cdot E_{\text{edu}} + 0.15 \cdot I$$
$$\text{Final Scorecard} = 0.55 \cdot \text{ATS} + 0.15 \cdot \text{Coverage} + 0.10 \cdot \text{Impact} + 0.10 \cdot \text{Format} + 0.10 \cdot \text{DomainFit}$$

**Numerical Calculation Example:**
• $F = 87.5\%$, $E_{\text{exp}} = 75.0\%$, $S = 80.0\%$, $E_{\text{edu}} = 100.0\%$, $I = 60.0\%$.
• $\text{ATS} = (87.5 \times 0.20) + (75 \times 0.25) + (80 \times 0.30) + (100 \times 0.10) + (60 \times 0.15) = 17.5 + 18.75 + 24.0 + 10.0 + 9.0 = \mathbf{79.25 \rightarrow 79\%}$.

## 12. COMPANY MATCH / JOB ROLE MATCHING

**Company Readiness Engine (`client/src/lib/readiness-engine.ts`):**
Evaluates candidate readiness across tech companies (e.g., Google, Amazon, TCS, Startups) and job roles using three weighted dimensions:
$$\text{Readiness Score} = 0.30 \times \text{Profile Match} + 0.30 \times \text{Skill Coverage} + 0.40 \times \text{Assessment Performance}$$

• **Profile Match (30%):** Weighted skill match against required + preferred skills.
• **Skill Coverage (30%):** Percentage of essential required skills matched.
• **Assessment Performance (40%):** Candidate's quiz score on relevant skill topics.

## 13. RECOMMENDATION ENGINE

**Recommendation Logic:**
• **Project Recommender (`client/src/lib/project-recommender.ts`):** Ranks target portfolio projects from `projects.json` based on missing skill coverage. High-priority missing skills count double.
• **Roadmap Generator (`client/src/lib/roadmap.ts`):** Maps missing domain skills to structured learning stages (Fundamentals, Advanced, Portfolio Project) from `roadmap.json`.
• *Deterministic Mapping:* Fully data-driven rule engine without random hallucinatory output.

## 14. AI / DATA SCIENCE CONTRIBUTION

**CRITICAL INTERVIEW DEFENSE: IS THIS REALLY AN AI PROJECT?**

**Q1: Is this really an AI project?**
**Answer:** "Yes, it belongs to the domain of **Symbolic AI and Natural Language Processing (NLP)**. AI encompasses both deterministic rule-based expert systems and statistical machine learning. Our system uses NLP text cleaning, entity extraction, pattern heuristics, and vector-like scoring algorithms."

**Q2: Did you train your own Machine Learning model?**
**Answer:** "No. We deliberately chose a rule-based NLP parser over an ML model like BERT or LLMs for three key reasons: 1) 100% deterministic accuracy with zero hallucinations, 2) sub-50ms execution latency, and 3) full explainability of scoring metrics for candidates."

**Q3: What algorithm did you use?**
**Answer:** "We implemented a custom pipeline combining Regex Token Normalization, Canonical Alias Dictionary Matching, Weighted Multi-Factor Linear Scoring, and Set Difference Priority Classification."

## 15. COMPLETE ARCHITECTURE

**System Layered Architecture:**
1. **Presentation Layer (Frontend):** React 19 SPA, Tailwind CSS glassmorphism UI, Wouter router, Lucide icons, Recharts SVG charts.
2. **Client State & Engine Layer:** Zustand global state stores (`resume-store`, `job-store`, `assessment-store`), PDF.js client text extractor.
3. **API Gateway / Service Bridge:** Fetch API with Bearer token authentication headers targeting `http://localhost:8000/api`.
4. **Application Server (Backend):** FastAPI framework, Pydantic data schemas, Dependency Injection (`Depends(get_db)`).
5. **NLP & Business Logic Layer:** `resume_parser.py` (PyMuPDF), `skill_extractor.py`, `resume_analyzer.py`, `jd_analyzer.py`.
6. **Persistence Layer (Database):** SQLAlchemy ORM, SQLite database (`server/career_compass.db`) storing Users, Resumes, Analyses, JDs, and Assessments.

## 16. DATA FLOW

**Data Transformation Objects:**
• *Raw PDF File (Binary ArrayBuffer)* $\rightarrow$
• *Clean Text String* (`"experienced in react python mysql..."`) $\rightarrow$
• *Extracted Skill Array* (`["MySQL", "Python", "React"]`) $\rightarrow$
• *Analysis Result JSON:*

```
{
"ats": { "overallScore": 82, "keywordCoverage": 75, "sectionScores": { "format": 90, "experience": 80,
"skills": 85, "education": 100, "impact": 65 } },
"skillsDetected": ["FastAPI", "Python", "React", "SQLAlchemy", "TypeScript"],
"bestFitDomain": "Full Stack Development",
"confidence": 88
}
```

## 17. STATE MANAGEMENT

**Zustand Store Architecture (`client/src/state/`):**
• **`auth-store.ts`:** Persists `accessToken`, `user` object, and login state in `localStorage` under key `"career-compass-auth"`.
• **`resume-store.ts`:** Stores current `parsedText`, `skillsDetected`, `atsScore`, and `analysis` state.
• **`job-store.ts`:** Holds active `jdAnalysis`, candidate match results (`matchingSkills`, `missingSkills`, `jobMatchScore`).
• **`assessment-store.ts`:** Tracks quiz attempt history, active question queue, and skill-wise correctness scores.
• **TanStack React Query:** Manages server synchronization, caching, and auto-refetching.

## 18. ROUTING

| Route Path | Component | Purpose & Key Action |
|---|---|---|
| /login | Login.tsx | User authentication & JWT token retrieval |
| /register | Register.tsx | New user registration & account setup |
| /dashboard | Dashboard.tsx | Main overview metrics, overall score, quick links |
| /upload | ResumeUpload.tsx | Drag-and-drop PDF upload & text parsing |
| /ats | AtsDashboard.tsx | ATS score breakdown & section radar metrics |
| /job-analysis | JobAnalysis.tsx | Paste JD text or select predefined job role |
| /skills | SkillGapAnalysis.tsx | Candidate vs JD match, missing skill priority matrix |
| /assessment | Assessment.tsx | Interactive technical quiz for skill validation |
| /analytics | Analytics.tsx | Historical attempt trends & skill improvement charts |
| /roadmap | LearningRoadmap.tsx | Step-by-step domain learning stages & timelines |
| /readiness | Readiness.tsx | Company readiness scores across target employers |

## 19. VISUALIZATION

**Recharts Components (`client/src/components/charts/`):**
• **`ats-score-breakdown.tsx`:** BarChart rendering section scores (Format, Experience, Skills, Education, Impact).
• **`domain-fit-radar.tsx`:** RadarChart comparing candidate skills against target domain requirement vectors.
• **Line Charts (`analytics.tsx`):** Tracks score progression across sequential assessment attempts over time.

## 20. ERROR HANDLING

**Robust Exception Handling Matrix:**
• **Non-PDF File Upload:** Frontend restricts input to `.pdf`; Backend validates content-type and extension, returning `400 Bad Request` ("Only PDF files are supported").
• **File Size >10MB:** Backend checks byte length before processing, raising `413 Payload Too Large`.
• **Scanned / Image PDF:** System catches empty parsed text post-processing, raising `400 Bad Request` informing user of missing OCR layer.
• **Corrupted / Password-Protected PDF:** `fitz.open()` exceptions and `doc.is_encrypted` checks return immediate `400 Bad Request`.
• **UI Feedback:** Integrated with `sonner` / `toaster` notification banners displaying actionable user messages.

## 21. TECHNICAL PROBLEMS AND DEBUGGING

**5 Real Engineering Challenges Resolved:**

**1. Skill Name Alias Mismatch (`React` vs `React.js` vs `ReactJS`):**
• *Problem:* Resumes had "React.js", JDs had "ReactJS", causing failed exact string matches.
• *Root Cause:* Raw string equality missing canonical normalization.
• *Fix:* Implemented alias mapping array in `skills.json` and regex character normalization in `skill_extractor.py`.

**2. Multi-Word Skill Truncation (`Spring Boot`, `Machine Learning`):**
• *Problem:* Naive unigram tokenization split "Spring Boot" into "Spring" and "Boot".
• *Root Cause:* Word-based splitting destroyed multi-word entities.
• *Fix:* Evaluated full multi-word phrase patterns against normalized text before fallback unigram checking.

**3. Scanned PDF Text Extraction Failure:**
• *Problem:* Image-only PDFs returned blank strings, causing silent score zeroes.
• *Root Cause:* Lack of embedded font vector layer in scanned PDFs.
• *Fix:* Added post-extraction text length checks raising explicit HTTP 400 validation error informing candidates.

**4. Zustand State Hydration Desynchronization:**
• *Problem:* Uploading a new resume did not clear previous JD match state on page navigation.
• *Root Cause:* Independent Zustand stores without reset subscriptions.
• *Fix:* Added explicit store reset action triggers on upload completion in `resume-store.ts`.

**5. CORS Block on FastAPI Server during Vite Dev:**
• *Problem:* Browser blocked API requests from `localhost:5000` to `localhost:8000`.
• *Root Cause:* Missing origin headers in FastAPI CORSMiddleware.
• *Fix:* Configured `allow_origins=["http://localhost:5000", "http://localhost:5173"]` in `server/main.py`.

## 22. PERFORMANCE

**Performance Optimization & Scale Benchmarks:**
• **Current Latency:** PDF Text Extraction (~15ms) + NLP Skill Normalization (~8ms) + Scoring (~5ms) = **Total Response Time ~28ms**.
• **Bottlenecks at Scale:** Synchronous PDF parsing in main ASGI event loop during concurrent multi-user uploads.
• **Capacity Analysis:**
- *100 Users:* Handled single-instance FastAPI + Uvicorn effortlessly (~2% CPU usage).
- *1,000 Users:* Requires Gunicorn process manager with 4 Uvicorn worker instances.
- *10,000 Users:* Requires Redis queue + Celery async background workers to offload PDF parsing.

## 23. SECURITY

**Current Security Implementation vs. Production Roadmap:**
• **Current:** JWT Token Authentication (Bearer scheme), Passlib with Bcrypt password hashing, Strict File Extension & MIME Type verification, 10MB payload size limits.
• **Production Additions:** Presigned S3 upload URLs to bypass application server bandwidth, ClamAV anti-virus file scanning buffer, Rate Limiting (SlowAPI), Data anonymization / PII scrubbing (phone/email stripping before DB storage), HTTPS with TLS 1.3 encryption.

## 24. PRODUCTION ARCHITECTURE

**Proposed Enterprise Production Architecture:**
Cloudflare CDN $\rightarrow$ API Gateway (Kong / NGINX) $\rightarrow$ FastAPI Stateless Microservices (Docker / Kubernetes) $\rightarrow$ RabbitMQ / Celery Worker Queue $\rightarrow$ PyMuPDF Async Parsing Nodes $\rightarrow$ PostgreSQL Database + Redis Cache + AWS S3 Storage.

## 25. SCALABILITY

**Horizontal Scaling Strategy:**
• **Stateless API Nodes:** FastAPI server instances maintain no session state in memory; all state is persisted in JWT tokens and database.
• **Database Optimization:** B-Tree indexing on `user_id`, `created_at`, and `resume_id` foreign keys.
• **Caching:** Redis cache for static `skills.json` and canonical domain mappings.

## 26. IF I HAD TO ADD REAL LLM/GENAI

**GenAI & Semantic Upgrade Roadmap:**
• **Current Keyword Matching:** Exact string match after normalization.
• **Semantic Vector Match (SentenceTransformers / OpenAI Embeddings):** Computes cosine similarity between candidate experience vectors and JD vectors. (e.g., Maps "Built RESTful endpoints in Spring Boot" $\approx$ "Backend API Development" with score 0.89).
• **LLM RAG Feedback (GPT-4o / Llama-3):** Generates personalized bullet-point rewriting suggestions and mock interview practice questions based on exact skill gaps.

## 27. WHY THESE TECHNOLOGIES?

**Interview Technical Justifications:**
• **Why React 19 + TypeScript?** Enables component reusability, modular chart rendering, and strict compile-time type safety.
• **Why FastAPI + Python?** Sub-millisecond performance via Starlette/Pydantic, combined with Python's rich ecosystem for text processing.
• **Why PyMuPDF over PyPDF2?** PyMuPDF is written in C (MuPDF), executing PDF extraction up to 10x faster with superior text layout fidelity.
• **Why Rule-Based over Pure LLM?** Instant execution (28ms vs 3000ms), zero cost per API call, and 100% deterministic explainability.

## 28. INTERVIEW CROSS QUESTIONS (TOP SELECTIONS)

| Category | Question & Professional English Answer | Tanglish Summary & Follow-Up |
|---|---|---|
| Easy / Core | **Q: How does PDF text extraction work in your project?** A: We use a dual approach: PDF.js on the client for instant browser preview, and PyMuPDF (fitz) on the backend server for robust parsing of raw bytes. | **Tanglish:** Client PDF.js-u preview kaaga, backend PyMuPDF fast text extract-panna use aagudhu. **Follow-Up:** What if PDF has password? We check `doc.is_encrypted` and throw 400 error. |
| Medium / NLP | **Q: How do you handle skill alias variations like React vs React.js?** A: We normalize text using regex `[^a-z0-9+.#\s-]` and match keywords against a canonical alias database in `skills.json`. | **Tanglish:** Text lowercase-ah maathi special characters clean panni, `skills.json` alias list oda match pannoom. **Follow-Up:** How do you prevent partial word false hits? Boundary matching and full phrase priority. |
| Deep Tech / Alg | **Q: Explain your ATS Scoring formula.** A: ATS score is a weighted linear combination: Format (20%), Experience (25%), Skills (30%), Education (10%), and Impact (15%). | **Tanglish:** 5 parameters ku weightage set panni, average weighted score calculate pannoom. **Follow-Up:** Why 30% for skills? Skills correlate highest with JD match. |
| AI / ML | **Q: Is there a trained ML model in your project?** A: No, it is a deterministic Symbolic AI system using NLP regex normalization and heuristic vector scoring. | **Tanglish:** Machine learning weights training panna la; rule-based NLP algorithms dhaan use pannoom. **Follow-Up:** How to add AI? Add SentenceTransformer embeddings. |
| Architecture | **Q: How would you scale this to 100,000 users?** A: Offload PDF parsing to Redis + Celery workers, use Gunicorn with multiple Uvicorn workers, and cache skills data. | **Tanglish:** API nodes stateless-ah maathi, background worker queue and Redis cache containerize pannoom. **Follow-Up:** Database choice? Migrate SQLite to PostgreSQL. |

## 29. 'I BUILT THIS' DEFENSE

**Personal Contribution Defense Strategy:**

• **My Personal Contributions:** I architected the full-stack system, designed the database schema in SQLAlchemy, implemented the PyMuPDF text parser (`server/services/resume_parser.py`), built the 5-signal ATS scoring engine (`resume_analyzer.py`), and developed the Zustand state-driven frontend components in React.

• **Hardest Technical Bug:** Resolving multi-word skill extraction without word boundary truncation. Fixed by evaluating multi-word phrase keys before unigram dictionary checks.

• **Verification:** Verified accuracy using test scripts (`test_phase2_backend.py`, `test_phase3_backend.py`) validating scoring edge cases.

## 30. COMPLETE PROJECT PITCH

**5-Minute Deep Technical Pitch:**

'Career Compass AI is an end-to-end resume intelligence platform engineered to bridge candidate skill gaps using rule-based NLP, weighted scoring algorithms, and dynamic career roadmaps. When a user uploads a PDF, PyMuPDF parses raw bytes in under 15ms. The text is passed through a regex normalization pipeline that strips noise while preserving tech symbols like C++ or Node.js. Our skill extractor cross-references text against a canonical database of over 100 skills and alias mappings. On the JD side, our parser uses contextual heuristics to classify skills into required versus preferred. We then run a candidate-vs-JD set matching engine that computes a weighted ATS score across 5 core dimensions, a skill coverage percentage, and domain fit metrics displayed on interactive Recharts visualizations. Finally, missing skills dynamically generate personalized project recommendations and learning roadmaps.'

## 31. FINAL REVISION CHEAT SHEET

| Core Workflow | 10 Must-Remember Technical Points | 10 Likely Interview Questions |
|---|---|---|
| Upload PDF $\rightarrow$<br>PyMuPDF Extract $\rightarrow$<br>Regex Normalize $\rightarrow$<br>Skill Extract $\rightarrow$<br>JD Classify $\rightarrow$<br>Set Match $\rightarrow$<br>ATS Score $\rightarrow$<br>Roadmap Render | 1. PyMuPDF fast C engine<br>2. PDF.js client preview<br>3. Regex `[^a-z0-9+.#\s-]`<br>4. Canonical `skills.json`<br>5. ATS 5-factor weighting<br>6. Required vs Preferred<br>7. Set match $O(N+M)$<br>8. Zustand global stores<br>9. FastAPI async ASGI<br>10. Symbolic Rule NLP | 1. Why PyMuPDF over PyPDF2?<br>2. How is ATS score computed?<br>3. How are aliases handled?<br>4. Is this an ML project?<br>5. How to handle scanned PDFs?<br>6. Why FastAPI over Express?<br>7. How does JD parsing work?<br>8. How to scale to 10k users?<br>9. Security measures used?<br>10. How to add GenAI RAG? |